

Code Assessment of the Solver7702Delegate Smart Contracts

PUBLIC

Date [June 11, 2026](#)

Produced for



CoW Protocol

By



CHAINSECURITY



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	6
5	System Considerations	7
6	Terminology	9
7	Findings	10
8	Limitations and use of report	13

1 Executive Summary

Dear CoW Protocol team,

Thank you for trusting us to help Cowswap with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Solver7702Delegate according to [Scope](#) to support you in forming an opinion on their security risks.

Cowswap implements `Solver7702Delegate`, an ERC-7702 delegation target for CoW Protocol solver EOAs that forwards calls from a fixed set of approved callers to arbitrary targets.

The most critical subjects covered in our audit are access control, functional correctness, and protocol integration. Access control is enforced through a fixed set of five immutable approved caller addresses; security regarding access control is high. Security regarding functional correctness of the call-forwarding mechanism is also high. Informational findings such as [Nonce Invalidation via Reverted Self-Call](#) have been addressed in the latest version. Security regarding protocol integration is high, though solvers upgrading to this delegate should be aware of a number of operationally relevant considerations; see [ERC-7702 Behavioral Changes for Solver EOAs](#) and [Loss of Compatibility with Protocols Relying on tx.origin](#).

The general subjects covered are code quality and operational security. The contract is minimal and focused; security regarding both is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Solver7702Delegate repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	1 June 2026	acde4b0cd452d208c2ea0f0fbbf83e3698decbf8	Initial version
2	8 June 2026	d2382297c6edee05b035ddcb285185d0938e038c	Final version

For the Solidity smart contracts, the compiler version `0.8.34` was chosen.

```
src/  
└─ Solver7702Delegate.sol
```

2.1.1 Excluded from scope

Any file not explicitly listed in the Scope section is out of scope. In particular, the overall CoW Protocol, the deployment scripts, and the tests are excluded from the scope.

3 System Overview

This system overview describes version (`Version 2`) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to improve readability.

Cowswap provides `Solver7702Delegate` as an ERC-7702 delegation target for CoW Protocol solver EOAs.

In CoW Protocol, batch settlements are submitted to `GPv2Settlement.settle()`. The settlement contract checks `msg.sender` against an on-chain solver allowlist. Only registered solver EOAs may call `settle`.

Under ERC-7702, an EOA can delegate its code slot to a deployed contract. Subsequent calls to that EOA execute the delegate's code in the EOA's own context, with the same address, balance, and storage.

3.1 Solver7702Delegate

`Solver7702Delegate` stores five addresses as `private immutable` values (`APPROVED_CALLER_0` through `APPROVED_CALLER_4`), set at construction and fixed thereafter. The contract has no storage, no owner, and no upgrade path.

Calls from approved addresses are forwarded to an arbitrary target encoded as the first 20 bytes of calldata, with the remaining calldata and all ETH forwarded to that target. Return and revert data are bubbled back. The target receives a `CALL`, not a `DELEGATECALL`, and executes in its own context.

Any sender can send ETH to the contract without calldata. Calls from unapproved addresses that carry no ETH revert with `Unauthorized(msg.sender)`.

4 Trust Model

4.1 Deployment Assumptions

The contract takes effect only when the solver EOA has an active ERC-7702 delegation pointing to this deployed instance. Clearing or replacing the delegation deactivates the contract.

The five approved caller addresses are fixed at construction. Rotating them requires deploying a new contract and re-delegating the solver EOA. Unused slots may be set to `address(0)` ; as `msg.sender` is never `address(0)` in normal EVM execution, such slots remain inert.

4.2 Roles and Privileges

4.2.1 EOA

The solver EOA that installs the ERC-7702 delegation.

Trust Level: Fully trusted.

Capabilities: The EOA controls which contract it delegates to by signing ERC-7702 authorization tuples. It can replace or revoke the delegation at any time, which immediately changes or removes the code executed on its behalf.

4.2.2 Approved Callers

The five addresses set at construction (`APPROVED_CALLER_0` through `APPROVED_CALLER_4`).

Trust Level: Fully trusted.

Capabilities: Each approved caller can perform any call in the name of the EOA to any target address with any calldata and any ETH value. A compromised approved caller can make the solver EOA interact with arbitrary contracts, including submitting settlements with arbitrary parameters to `GPv2Settlement` . However, they cannot sign ERC-7702 authorization tuples and therefore cannot change the delegation itself.

4.3 Upgradeability

The contract is non-upgradeable. The approved callers are fixed at construction and there are no setters.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Accumulated ETH Only Recoverable by Solver Operator

System Consideration	Version 1
The <code>Solver7702Delegate</code> fallback accepts ETH from any sender. <code>_callThrough()</code> forwards only the ETH of the current call via <code>callvalue()</code> ; ETH previously accumulated in the solver's balance cannot be extracted by approved callers. Moving the accumulated balance requires the solver operator to sign a transfer directly. If the solver's private key is lost or was ephemeral, any accumulated ETH is permanently stuck.	

SC2 ERC-7702 Behavioral Changes for Solver EOAs

System Consideration	Version 1
The following is a non-exhaustive list of behavior changes that solvers must be aware of when adopting ERC-7702 delegation:	
L2 address aliasing of ERC-7702 delegated EOAs. L2 bridges such as Arbitrum alias the <code>msg.sender</code> of L1-to-L2 messages when the sender is a smart contract. A delegated EOA is classified as a contract by bridge infrastructure, causing <code>msg.sender</code> on L2 to be an offset address that the solver operator does not control. Any solver relying on <code>msg.sender</code> matching the original EOA address may lock funds.	
ERC-1271 signature routing via <code>signer.code.length</code>. Contracts that rely on <code>signer.code.length == 0</code> to decide between <code>ecrecover</code> and an ERC-1271 <code>isValidSignature</code> call will revert once the solver EOA has delegated to the <code>Solver7702Delegate</code> , as the <code>Solver7702Delegate</code> does not implement ERC-1271.	

SC3 Losses in Case of Caller Key Compromise

System Consideration	Version 1
The approved caller keys have full control over the solver. A compromise can lead to the following losses, among others:	
- Any non-native assets held by the solver can be drained, though as a warm wallet it is not expected to hold significant assets. - Malicious batches can be submitted, settling users at their limit price and extracting the surplus. This could also lead to slashing by CoW Protocol. - Any assets held by the approved caller can be drained. If an automated ETH refill system is in place that tops up the ETH balance of the solver after it goes below a certain threshold, the refill system must implement its own rate limits. Otherwise a compromised caller could be repeatedly drained and refilled.	

Cowswap added the following:

As the solver is allowlisted on the settlement contract, without needing to settle any orders in particular, any funds stored in the settlement contract ("buffers") can be drained using the EOA delegation. As the settlement can at times hold hundreds of thousands of dollars worth of assets, these losses could be substantial and the

solver remains responsible for any lost funds as part of their agreement with the protocol. There has been an incident related to this in the past. <https://cow.fi/learn/cow-swap-solver-exploit-post-mortem>

SC4 Silent Success After Delegation Revocation

System Consideration	Version 1
----------------------	-----------

An ERC-7702 delegation can be revoked at any time by the solver EOA owner by submitting a new authorization tuple with a zeroed target. Once the delegation is removed, the solver EOA reverts to a plain EOA with no code. In the EVM, calls to accounts with no code always return success regardless of the calldata or value sent.

After revocation, any call by an approved caller will succeed silently with no effect. Approved callers must implement off-chain mechanisms to detect these revocations.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

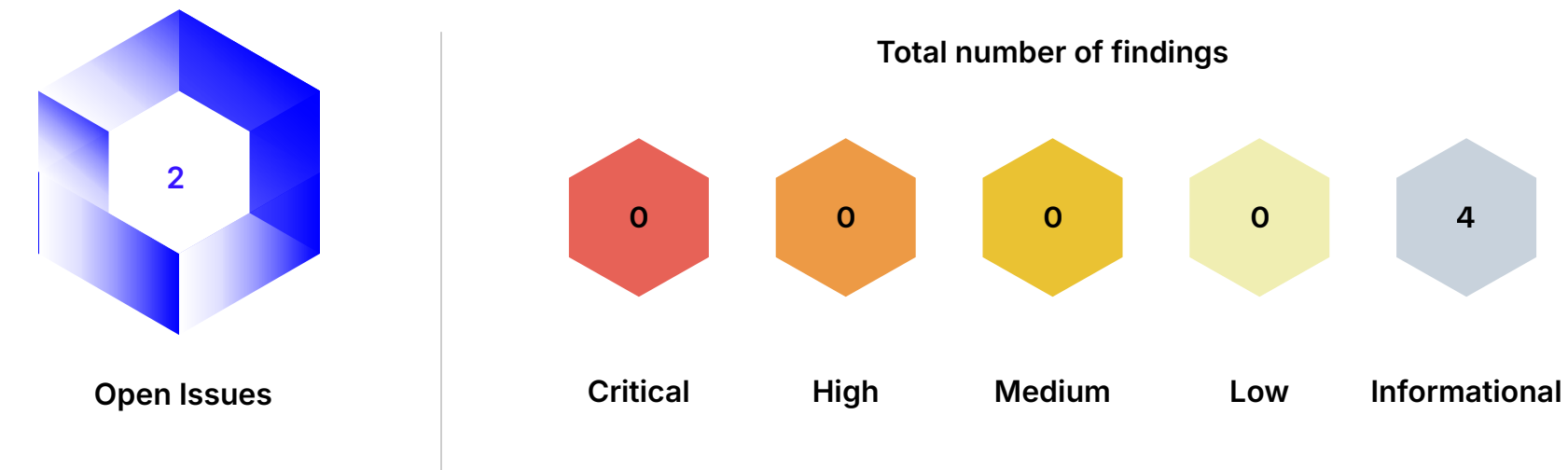
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the [Terminology](#) section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Info	#001	Misleading Memory Safety Code Comment	Code Corrected	✓
Info	#002	Nonce Invalidation via Reverted Self-Call	Code Corrected	✓
Info	#003	Phantom Function Allows Silent No-Op on ETH-Carrying Calls	Acknowledged	⊖
Info	#004	Loss of Compatibility with Protocols Relying on Tx.origin	Acknowledged	⊖

7.2 Descriptions

#001 Misleading Memory Safety Code Comment

Informational	Version 1	Code Corrected 
---------------	-----------	--

The inline assembly block in `_callThrough()` includes the comment: *"We take full control of memory in this inline assembly block because it will not return to Solidity code. This is why we overwrite the Solidity scratch pad at memory position 0."* This comment is misleading:

The scratch space (`0x00 -- 0x3f`) is always safe to overwrite per Solidity's memory safety specification; more notably, the free memory pointer (`0x40`) and the zero slot (`0x60`) are also written to. This has no effect in this function, since the code block never returns to Solidity.

Code Corrected

The misleading comment in `_callThrough()` of `Solver7702Delegate` has been updated to accurately describe the memory locations that may be overwritten (i.e. starting at `0x00` and extending into the free memory pointer and zero slot when the calldata is large enough). It also clarifies that this has no adverse effect since the assembly block never returns to Solidity.

#002 Nonce Invalidation via Reverted Self-Call

Informational	Version 1	Code Corrected 
---------------	-----------	--

Solvers can send a transaction to themselves with zero value and empty calldata to bump their nonce and invalidate pending transactions. With an ERC-7702 delegation active, such a call enters the fallback and reverts with `Unauthorized(msg.sender)` because the EOA's own address is not among the approved callers and `msg.value` is zero. Even though the transaction reverts, the EVM still increments the EOA's nonce so pending transactions are still invalidated. However, solvers must not expect this transaction to succeed and should submit the transaction even when simulation fails.

Code Corrected

The `fallback()` of `Solver7702Delegate` has been updated to gracefully return when `msg.sender` is the delegated EOA itself, so a self-call used to cancel a pending transaction succeeds.

#003 Phantom Function Allows Silent No-Op on ETH-Carrying Calls

Informational	Version 1	Acknowledged 
---------------	-----------	--

For a call from a non-approved caller where `msg.value > 0`, the `Solver7702Delegate` fallback returns success regardless of the calldata supplied, ignoring it entirely. Calls from approved callers are not affected, as they are always forwarded to the encoded target via `_callThrough()` whether or not ETH is sent.

A caller invoking a named function on a contract address would typically expect the call to either revert (if the function is not implemented) or succeed with the intended effect (if it is). Here, neither happens: the call succeeds but has no effect. This pattern is analogous to the phantom function vulnerability known from the WETH contract, where any call to `permit()` succeeds silently because `permit` is not implemented and the call falls through to the receive fallback with no effect. The key difference is that this variant additionally requires `msg.value > 0`, making it less likely to be triggered unintentionally.

In the worst case, if the solver holds standing token approvals to a contract with pull capability, that contract could interpret the phantom success as authorization and transfer tokens out of the solver.

Acknowledged

Cowswap has decided to keep the code unchanged and provides the following reasoning for their design decision:

The primary reasoning for this system design is to try and preserve *as much* of the EOA behavior as reasonably possible. In particular, it is *explicitly necessary* to be able to receive native token transfers, as the solver is the address we send their payouts (in native token). Sending native tokens is a relatively anomalous event as you note here, so the risk seems low. However, to prevent confusion when a signer attempts to send a transaction on a misconfigured EIP-7702 delegation/contract, it would be helpful to have an explicit error to mark this.

#004 Loss of Compatibility with Protocols Relying on Tx.origin

Informational

Version 1

Acknowledged 

Some protocols require `tx.origin == msg.sender` to restrict interaction to direct EOA senders. A plain solver EOA satisfies this naturally. After upgrading to this delegate, transactions are submitted by an approved caller, making `tx.origin` the approved caller's address while `msg.sender` in the forwarded call is the solver EOA. Solvers that integrate with smart contracts expecting `tx.origin` to match the solver EOA may therefore lose compatibility.

Acknowledged

The client acknowledges this behavioral change and will document it in the solver integration guide so that affected solvers are aware of it before delegating to the `Solver7702Delegate`.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.